SSUET RideShare — Operations handbook

For	the SSUET ride-share office — the people who run it, not the people who wrote it
Version	1.0.0 · build 15 · 7 November 2026
Covers	how to start the system, keep it alive, and what to do when something breaks

Every command in this handbook was run against the real deployment before it was written down.

1. What you are running

A car- and bike-pooling service for Sir Syed University students and staff: students publish journeys, request seats, travel together, chat in the app, rate each other, and report problems. An administrator console handles verification, moderation and analytics.

Members use	the Android app (Expo build), point it at `https://rideshare.ssuet.edu.pk/api/v1`
The office uses	`https://rideshare.ssuet.edu.pk` in a browser
The API	`https://rideshare.ssuet.edu.pk/api/v1` — same origin, so no CORS surprises
Health probes	`/api/v1/health` (liveness, never touches the database) and `/api/v1/health/ready` (one query)
Deep health	`GET /api/v1/admin/health` with an admin token — the panel in §5

Everything runs on **one Linux server**. There is no separate worker machine, no message queue, and no third-party service required: if the server is up, the product is up.

Accounts that exist on a fresh install

Account	Role	What it is for
`admin@example.com`	Administrator	the office console — **change this password on day one**

Demo members (`ali@` , `fatima@` , `ahmed@` , `drfaheem@` , `usman@` `@ssuet.edu.pk`) exist only in a database loaded with the seed script. A production database normally has none.

2. Day one — do these in order

Each step takes minutes and ends with something you can see. If a step fails, stop: the one after it depends on it.

1. **Log in to the console.** `https://rideshare.ssuet.edu.pk` → the admin account. You should land on *Operations overview* with real numbers, not zeros.
2. **Change the administrator password** in *Settings*, and record it in the office's password manager. Never in a chat message.
3. **Run the smoke test** from the server:

   ```
   cd /srv/ssuet-rideshare && ./deploy/smoke.sh https://rideshare.ssuet.edu.pk
   ```

   Expect `17 passed, 0 failed`. This checks TLS, health, readiness, security headers, the console shell, an unauthenticated API call, the upload cap and rate limiting.
4. **Open the deep health panel** (§5) and confirm `status: ok`, `schema_revision` equal to `schema_head`, and an empty `warnings` list.
5. **Check the first backup exists** — `ls -la /var/backups/ssuet-rideshare/` should show a dated directory with `database.sqlite3` (or `database.dump`), `media.tar.gz`, `SHA256SUMS` and `MANIFEST.txt`. If not, run `sudo -u ssuet ./deploy/backup.sh` by hand and read the error.
6. **Confirm the timers are armed**:

   ```
   systemctl list-timers | grep ssuet        # health check every 15 minutes
   crontab -l -u ssuet                       # expire rides, backup, media sweep
   ```

7. **Register one real account on a phone**, verify it, publish one journey, request it from a second account, and complete the trip. This is the only end-to-end test that proves the app's network path, push notifications aside.
8. **Write down** in the office log: the date, who did these steps, and the output of `./deploy/smoke.sh`. That page is the handover.

3. What is running where

Piece	What it does	Where it lives	How to see it is alive
nginx	TLS, one origin for console + API, upload cap, edge rate limits	`/etc/nginx/sites-enabled/ssuet-rideshare.conf`	`systemctl status nginx`, `nginx -t`
gunicorn (5 uvicorn workers)	the FastAPI application	systemd unit `ssuet-api`	`systemctl status ssuet-api`
PostgreSQL	members, rides, trips, reviews, reports, audit log	`ssuet_rideshare` database	`pg_isready`, `systemctl status postgresql`
Uploaded files	ID cards, vehicle papers, profile photos	`/var/lib/ssuet-rideshare/media`	`/api/v1/admin/media` or the health panel
Housekeeping	expires stale rides, sweeps orphaned uploads	cron, every 15 min / nightly	`/admin/health` → `housekeeping.last_run_at`
Backups	database + media with checksums	cron, 03:15 nightly, 14 days kept	`/var/backups/ssuet-rideshare/`
Health check	5 probes, exit code 0/1/2, optional webhook	timer, every 15 min	`journalctl -u ssuet-monitor -n 30`

Logs are the first place to look, always:

```
journalctl -u ssuet-api -n 100 --no-pager     # application log (request ids included)
tail -50 /var/log/nginx/error.log             # edge errors, TLS, rate limiting
journalctl -u ssuet-monitor -n 50             # the health checks and their verdicts
tail -30 /var/log/ssuet-backup.log            # last backup run
```

4. The routine

Every day (2 minutes)

```
cd /srv/ssuet-rideshare && ./deploy/monitor.sh
```

Five lines, each `OK` or `FAIL`: liveness (with a latency budget), readiness, certificate expiry, disk free, backup age. Exit code `0` healthy, `1` warning, `2` failure. If it says anything other than five `OK` lines, jump to §6.

Then glance at the console: *Verification Requests*, *Vehicle Verification*, *Reports & Complaints*. Queues that grow for a week are how trust quietly erodes.

Every week

1. **Verification queue to zero.** An unverified member cannot publish or request.
2. **Reports triaged.** Every report ends `RESOLVED` or `DISMISSED` with a note. The reporter is notified automatically; the note is what they would read if they asked.
3. **Storage panel** — *Settings* or `GET /api/v1/admin/media`: `files`, `bytes`, `orphans`. Orphans should be zero after the nightly sweep.
4. **Skim the audit log** of administrator actions. Nobody should be surprised by it.

Every month

1. **Prove a backup restores** (§7). An untested backup is a rumour.
2. **Update the server**: `apt-get update && apt-get upgrade` in a quiet window, then `./deploy/smoke.sh` again.
3. **Review administrator accounts** — one per person who needs it, no shared logins.
4. **Read the numbers** in `/admin/health` and the *Analytics* page, and decide whether anything needs a decision (more capacity, a policy change) rather than a fix.

---

5. The health panel (the single most useful command)

```
TOKEN=$(curl -s -X POST https://rideshare.ssuet.edu.pk/api/v1/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"admin@example.com","password":"…"}' | python3 -c 'import
json,sys;print(json.load(sys.stdin)["access_token"])')

curl -s -H "Authorization: Bearer $TOKEN" \
  https://rideshare.ssuet.edu.pk/api/v1/admin/health | python3 -m json.tool
```

What to look at, in order:

Field	Healthy	What a bad value means
`status`	`ok`	`degraded` = read `warnings[]` before anything else
`database.latency_ms`	under ~5 ms	tens of ms means the database is struggling
`schema_revision` vs `schema_head`	equal	behind head = a deploy skipped `alembic upgrade head`
`queues.*`	small, moving	a queue that only grows is a process problem, not a backlog
`media.disk_free_bytes`	> 10 % of total	below that, uploads start failing
`housekeeping.last_run_at`	within the last hour	`null` or hours old = the cron job died
`process.uptime_seconds`	hours or days	seconds = the service is crash-looping

Every response also carries `X-Request-ID`. If a member sends you a screenshot of an error, that id matches the exact log line in `journalctl -u ssuet-api`.

---

6. When something is wrong

Work top-down. Each playbook starts with the cheapest possible check.

The console or the app will not load

1. `./deploy/monitor.sh` — is it liveness, readiness, or both?
2. Both failing → the API is down: `systemctl status ssuet-api`, then `journalctl -u ssuet-api -n 80`. A configuration error in `api.env` shows up here plainly (missing `JWT_SECRET_KEY`, bad `DATABASE_URL`).
3. Liveness fine, readiness failing → the **database** is unreachable: `pg_isready`, `systemctl status postgresql`, check disk space (§disk).
4. Liveness fine on the server but not from your laptop → **nginx or TLS**: `nginx -t`, `systemctl status nginx`, check the certificate dates.

Requests are slow

1. `/admin/health` first: `database.latency_ms` above ~20 ms means the database, not the app. Check `pg_stat_activity` for a long-running query or a lock.
2. Database fast but the app slow → look at the log for `SLOW …` lines (anything over `SLOW_REQUEST_MS`, default 500 ms) and note the request id. A single slow endpoint is a code question; everything slow is the server (CPU, disk, memory).
3. Measured capacity on a 2-vCPU host is roughly **145 requests per second** for the busiest screen. If the office sees traffic near that, the answer is a bigger server, not a code change.

Disk is filling up

```
df -h /var/lib/ssuet-rideshare /var/backups/ssuet-rideshare
du -sh /var/lib/ssuet-rideshare/media /var/backups/ssuet-rideshare
```

Profile photos are ~15 KB each after normalisation; **documents are stored at full size on purpose** (an ID card has to stay readable). Backups are the usual culprit — 14 daily copies of the database and the media archive. Free space by removing the oldest backup directories, never the newest.

The certificate is about to expire

`monitor.sh` warns below 14 days. Renew with `certbot renew` (the shipped nginx config already serves the ACME challenge path), then `systemctl reload nginx` and re-run `./deploy/smoke.sh`.

A member is being harassed, or a photo is wrong

1. **Reports & Complaints** in the console. Reports arrive with the reporter, the member, the category, and — for photos — the exact picture that was reported.
2. Handle it: **under review** while you check, then **resolve** or **dismiss** with a note. The reporter is notified automatically.
3. For a profile photo, tick **"Also remove the photo"** in the same decision. The file is kept as evidence for administrators and the member is told.
4. For behaviour that is not about a photo, resolving the report does **not** suspend anyone. Suspension is a separate, explicit action on the member's dossier — do it deliberately, with the reason recorded.

A journey is stuck

Members can cancel; rides expire automatically every 15 minutes after departure (`scripts.expire_stale_rides`). If a *trip* is stuck because one side never confirmed, an administrator can resolve it from *Active Trips* — the audit trail records who did.

Spam or duplicate signups

Verification is the gate: an account with an unrecognised email domain stays `UNVERIFIED` until the office verifies the ID it submitted. Reject with a note, do not delete accounts — the audit trail matters more than a tidy member list.

---

7. Backups, restore, and the monthly proof

What a backup contains: a database dump, a tarball of the media volume, a `SHA256SUMS` file and a readable `MANIFEST.txt`. Nightly at 03:15, kept for 14 days.

```
sudo -u ssuet /srv/ssuet-rideshare/deploy/backup.sh          # take one now
ls -la /var/backups/ssuet-rideshare/ | tail -3
```

Restore (the monthly proof, or a real disaster):

```
sudo systemctl stop ssuet-api                    # restore refuses to run while the API is
live
sudo -u ssuet /srv/ssuet-rideshare/deploy/restore.sh /var/backups/ssuet-rideshare/<date>
sudo systemctl start ssuet-api
./deploy/smoke.sh https://rideshare.ssuet.edu.pk # expect 17 passed
```

The restore script verifies every checksum before touching anything, and refuses to continue if the member count in the dump looks implausible. Do this on a quiet evening and write the date in the office log.

---

8. Releasing a new version

The order below is deliberate: nothing is visible to members until the database it talks to already understands it.

```
cd /srv/ssuet-rideshare
sudo -u ssuet git pull
sudo -u ssuet backend/.venv/bin/pip install -r backend/requirements-prod.txt
sudo -u ssuet backend/.venv/bin/alembic upgrade head          # 1. database first
sudo -u ssuet backend/.venv/bin/alembic check                 # 2. models and migrations
agree
cd admin-dashboard && npm ci && npm run build                 # 3. console
sudo rsync -a --delete dist/ /srv/ssuet-rideshare/admin-dashboard/dist/
sudo systemctl reload ssuet-api                               # 4. application
cd /srv/ssuet-rideshare && ./deploy/smoke.sh https://rideshare.ssuet.edu.pk
```

If the smoke test fails, roll back the *application* with `git checkout <previous tag> && sudo systemctl reload ssuet-api`. A database migration is not automatically reversible — read the migration's `downgrade()` before a release that drops anything.

The mobile app is a separate release: `cd mobile && eas build -p android --profile production`, with `EXPO_PUBLIC_API_URL=https://rideshare.ssuet.edu.pk/api/v1` set at build time. Old app versions keep working against the new API — that is a compatibility rule the project has kept so far (see doc 03 §compatibility).

---

9. The numbers that matter

Limit	Value	Why
Verification / vehicle document upload	5 MB, JPG/PNG/WebP/PDF by content	an ID card has to stay legible
Profile photo upload	2 MB, JPG/PNG/WebP by content	a phone photo, not a poster
Photo pixel ceiling	40 megapixels	refuses decompression bombs before decoding
Profile photo stored as	512 px WebP, no EXIF/GPS; 128 px thumbnail on demand	privacy and mobile data
Uploads per member	10 per minute	stops a script from filling the disk
Photo reports per member	5 per minute, one open at a time	stops the queue being flooded
Login attempts	5 per minute per account	password guessing
Edge limit (all API traffic, per IP)	60 requests/second	protects the origin
Sessions	access token 30 minutes, refresh 30 days	long enough to be usable, short enough to matter
Slow-request logging	over 500 ms	what to look for when members say "it hangs"
Housekeeping	every 15 minutes	expired rides, verification windows
Orphan sweep retention	7 days	a file mid-flow is not garbage yet
Backups	nightly 03:15, keep 14	a fortnight covers a slow discovery

10. Who to call

Situation	First contact
Members, policy, verification decisions	the ride-share office
The service is down and §6 did not fix it	the developer on record for this deployment
Suspected data breach or lost device	the developer **and** the university IT office, same day
Server, DNS, certificate, billing	whoever holds the VPS account

Keep this table filled in with real names and phone numbers before the first member signs up. An escalation list with blanks in it is not an escalation list.

11. Where the detail lives

This handbook is the summary. Each line below is a document in the repository under `documentation/`, written for whoever inherits the work.

Document	Read it when
`01-architecture.md`	you need to know how the pieces fit
`02-database-design.md`	you are looking at data, or planning a change to it
`03-api-contracts.md`	you are integrating, or a client is misbehaving
`04-user-journeys.md`	you want to know what members actually do
`05-roadmap-and-progress.md`	you want the history of every phase
`06` – `12`	testing, mobile, admin console, standards, security
`13-admin-runbook.md`	day-to-day console questions in more depth
`14` – `16`	the release-candidate audit, media pipeline, vehicle documents
`17-production-deployment.md`	building the server from scratch
`18-observability-and-load.md`	the health panel, monitoring, measured capacity
`19-profile-photos.md`	how photos are stored and who may see them
`20-image-normalisation.md`	why a stored photo is not the file that was uploaded
`21-photo-moderation.md`	how a reported picture is handled and kept as evidence

Sign-off

Handover accepted by the ride-share office:

	Name	Role	Date	Signature
Prepared by		Developer		
Received by		Ride-share office		
Witnessed by		University IT		

Day-one checklist completed on: _____ Smoke test result: _____ / 17

First restore rehearsal completed on: _____ Result: _____

Rebuilding this document

The PDF is generated from this Markdown file, so there is one source of truth:

```
pip install markdown-it-py playwright && python -m playwright install chromium
python scripts/build_handbook_pdf.py        # → documentation/22-operations-handbook.pdf
```

Edit the Markdown, never the PDF.